

Pembagian Tugas Kelompok

Tugas 4 II4021 Kriptografi

Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi

1. Tujuan Pembagian Tugas

Pembagian tugas ini dibuat untuk membagi pekerjaan implementasi aplikasi password manager terdistribusi secara adil kepada tiga anggota kelompok, yaitu A, B, dan C. Karena tidak ada anggota yang memiliki kemampuan khusus yang sangat menonjol, pembagian dilakukan berdasarkan kombinasi jumlah pekerjaan, tingkat kesulitan, dependensi antarfitur, dan kebutuhan integrasi akhir.

Aplikasi yang akan dibuat berbasis CLI dengan arsitektur client-server. Proses kriptografi utama dilakukan di sisi client, sedangkan server berfungsi sebagai media penyimpanan data terenkripsi menggunakan SQLite. Sistem mendukung pembuatan vault, akses normal, akses backup, pengelolaan data password, serta fitur bonus kriptografi visual untuk recovery share.

2. Asumsi Implementasi

Aspek	Keputusan
Jumlah anggota	3 orang: A, B, C
Bentuk aplikasi	CLI-based application
Arsitektur	Client-server
Bahasa utama	Python
Server	Flask atau FastAPI
Database server	SQLite
Enkripsi vault	AES-128-GCM
Secret sharing	Shamir Secret Sharing skema (2,3)
KDF	PBKDF2, Argon2, atau scrypt
Password generator	CSPRNG
Fitur bonus	Diambil semua, yaitu QR code + visual cryptography untuk recovery share
Target internal	Implementasi selesai H-3, testing selesai H-2, laporan selesai H-1

3. Pembagian Peran Utama

Anggota	Peran Utama	Fokus Besar
A	Crypto Core Engineer	Implementasi kriptografi inti, Shamir Secret Sharing, AES-GCM, KDF, CSPRNG, dan bonus visual cryptography
B	Backend & Storage Engineer	Implementasi server, API, SQLite, penyimpanan vault terenkripsi, server share, nonce, dan metadata
C	Client CLI & Integration Engineer	Implementasi CLI, alur pengguna, CRUD password, mode normal, mode backup, integrasi akhir, testing, laporan, dan demo

Pembagian ini dibuat agar setiap anggota tetap memiliki beban teknis yang seimbang. A memegang bagian kriptografi yang kompleks, B memegang bagian server dan penyimpanan, sedangkan C memegang bagian CLI, integrasi fitur, dan validasi end-to-end. Namun, dokumentasi dan testing tetap dibagi ke semua anggota sesuai modul masing-masing.

4. Detail Tugas Anggota A — Crypto Core & Bonus

4.1 Tanggung Jawab Utama

A bertanggung jawab terhadap seluruh fungsi kriptografi inti yang digunakan oleh client. Bagian ini menjadi fondasi utama aplikasi karena seluruh fitur lain bergantung pada fungsi enkripsi, dekripsi, pembagian share, rekonstruksi master key, dan password generator.

4.2 Daftar Tugas Implementasi

Kode	Tugas	Output
A1	Membuat fungsi pembangkitan master key secara acak	Master key 128-bit atau sesuai kebutuhan AES-128
A2	Mengimplementasikan Shamir Secret Sharing skema (2,3)	Tiga share: local share, server share, recovery share

Kode	Tugas	Output
A3	Mengimplementasikan rekonstruksi master key dari dua share valid	Fungsi reconstruct_master_key()
A4	Mengimplementasikan AES-128-GCM untuk enkripsi dan dekripsi vault	Fungsi encrypt_vault() dan decrypt_vault()
A5	Mengimplementasikan KDF dari master password	Fungsi derive_key() dengan salt dan parameter KDF
A6	Mengimplementasikan enkripsi local share menggunakan key hasil KDF	Local share terenkripsi dan nonce local share
A7	Mengimplementasikan dekripsi local share	Local share berhasil dibuka jika master password benar
A8	Mengimplementasikan CSPRNG password generator	Password otomatis sesuai panjang input user
A9	Menentukan format share	Misalnya JSON atau base64 yang memuat koordinat dan nilai share
A10	Membuat validasi jika share salah atau rusak	Error handling saat rekonstruksi gagal
A11	Implementasi bonus: recovery share menjadi QR code	QR code berisi recovery share
A12	Implementasi bonus: visual cryptography skema (2,2) untuk QR code	Dua gambar share visual
A13	Implementasi bonus: penggabungan dua gambar share menjadi QR kembali	QR hasil gabungan dapat dipindai
A14	Membuat unit test untuk fungsi kriptografi	Test AES-GCM, SSS, KDF, CSPRNG, dan bonus

4.3 File yang Menjadi Tanggung Jawab A

Contoh struktur file:

```
client/crypto/aes_gcm.py
client/crypto/shamir.py
client/crypto/kdf.py
client/crypto/password_generator.py
client/crypto/visual_crypto.py
client/crypto/qr_recovery.py
tests/test_crypto.py
```

4.4 Bagian Laporan yang Dikerjakan A

A menulis bagian:

1. Teori singkat Shamir Secret Sharing.
2. Teori singkat AES-GCM.
3. Teori singkat KDF.
4. Teori singkat CSPRNG.
5. Teori dan implementasi bonus visual cryptography.
6. Penjelasan implementasi file kriptografi.
7. Hasil pengujian kriptografi.

4.5 Risiko Pekerjaan A

Risiko	Mitigasi
Rekonstruksi master key gagal karena format share tidak konsisten	Sepakati format share sejak awal dengan B dan C
AES-GCM gagal karena nonce/tag tidak disimpan benar	Buat struktur data ciphertext yang jelas
Bonus visual cryptography terlalu memakan waktu	Implementasi fitur utama dulu, bonus dikerjakan setelah core crypto stabil
Password generator tidak benar-benar CSPRNG	Gunakan library secure random seperti secrets di Python

5. Detail Tugas Anggota B — Backend, Server, dan SQLite

5.1 Tanggung Jawab Utama

B bertanggung jawab terhadap server dan database. Server tidak boleh melakukan proses kriptografi sensitif, tidak boleh menyimpan plaintext vault, tidak boleh menyimpan master key utuh, local share, recovery share, password pengguna plaintext, atau key turunan master password. Server hanya menjadi tempat penyimpanan data terenkripsi dan server share.

5.2 Daftar Tugas Implementasi

Kode	Tugas	Output
B1	Menentukan framework server	Flask atau FastAPI
B2	Membuat struktur database SQLite	File database dan schema table
B3	Membuat tabel user/metadata	Menyimpan identitas atau metadata user
B4	Membuat tabel vault	Menyimpan vault terenkripsi sebagai BLOB
B5	Membuat kolom untuk nonce vault	Nonce vault tersimpan
B6	Membuat kolom untuk server share	Server share tersimpan
B7	Membuat endpoint create vault	Menerima server share, vault encrypted, nonce, metadata
B8	Membuat endpoint get vault normal mode	Mengirim server share, vault encrypted, nonce ke client
B9	Membuat endpoint update vault	Menerima vault terenkripsi baru dan nonce baru
B10	Membuat endpoint health check server	Untuk simulasi server aktif/tidak aktif
B11	Membuat validasi agar server tidak menerima data terlarang	Reject master key, local share, recovery share, plaintext vault
B12	Membuat error handling API	Response jelas jika vault tidak ditemukan/server error
B13	Membuat dokumentasi endpoint sederhana	Untuk README dan laporan
B14	Membuat test backend dan database	Test penyimpanan BLOB, nonce, server share, metadata

5.3 Contoh Schema SQLite

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  created_at TEXT NOT NULL
```

);

```
CREATE TABLE vaults (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_id INTEGER NOT NULL,  
    encrypted_vault BLOB NOT NULL,  
    vault_nonce BLOB NOT NULL,  
    server_share TEXT NOT NULL,  
    updated_at TEXT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Schema bisa disesuaikan, tetapi server tetap hanya boleh menyimpan data terenkripsi dan data pendukung.

5.4 File yang Menjadi Tanggung Jawab B

Contoh struktur file:

```
server/app.py  
server/routes/vault_routes.py  
server/db/database.py  
server/db/schema.sql  
server/models.py  
server/config.py  
tests/test_server.py
```

5.5 Bagian Laporan yang Dikerjakan B

B menulis bagian:

1. Arsitektur client-server.
2. Peran server dan batasan zero-knowledge.
3. Rancangan database SQLite.
4. Penjelasan endpoint API.
5. Penjelasan penyimpanan vault terenkripsi sebagai BLOB.
6. Bukti bahwa server tidak menyimpan plaintext.
7. Hasil pengujian server dan database.

5.6 Risiko Pekerjaan B

Risiko	Mitigasi
Server tidak sesuai prinsip zero-knowledge	Buat validasi input dan dokumentasikan data apa saja yang boleh disimpan
Vault tidak tersimpan sebagai BLOB	Pastikan tipe kolom dan proses insert benar
Endpoint belum siap saat C mengerjakan CLI	Buat mock response atau dummy endpoint lebih awal
Format data tidak cocok dengan client	Sepakati format request/response API sejak awal

6. Detail Tugas Anggota C — CLI, Mode Akses, CRUD, Integrasi, Testing, dan Dokumentasi

6.1 Tanggung Jawab Utama

C bertanggung jawab terhadap aplikasi CLI yang digunakan oleh user, alur penggunaan aplikasi, integrasi crypto module dari A dengan server dari B, mode normal, mode backup, CRUD password, backup vault lokal, testing end-to-end, README, laporan final, dan video demo.

6.2 Daftar Tugas Implementasi

Kode	Tugas	Output
C1	Membuat struktur menu CLI	Menu create vault, open vault, add, edit, delete, list, backup mode
C2	Membuat alur pembuatan vault	Input master password, generate vault kosong, simpan local/server/recovery share
C3	Mengintegrasikan crypto module A pada CLI	CLI dapat memanggil AES, SSS, KDF, CSPRNG
C4	Mengintegrasikan server API B pada CLI	CLI dapat create, get, update vault via server
C5	Menyimpan local share terenkripsi di client	File local storage
C6	Menyimpan salt KDF dan nonce local share di client	File konfigurasi client
C7	Menyimpan backup vault lokal terenkripsi	Backup digunakan saat server tidak aktif
C8	Implementasi mode normal	Local share + server share untuk membuka vault
C9	Implementasi mode backup	Local share + recovery share + backup vault lokal
C10	Membatasi mode backup hanya untuk melihat data	Add/edit/delete disabled

Kode	Tugas	Output
C11	Implementasi tambah data password manual	Data masuk ke vault
C12	Implementasi tambah data password otomatis	Memanggil CSPRNG dari A
C13	Implementasi edit data password	Vault terenkripsi ulang setelah perubahan
C14	Implementasi delete data password	Vault terenkripsi ulang setelah penghapusan
C15	Memastikan nonce baru dibuat setiap update vault	Nonce lama tidak dipakai ulang
C16	Membuat integrasi testing end-to-end	Semua alur utama diuji
C17	Menyusun README	Cara menjalankan program dan dependensi
C18	Menggabungkan laporan dari A dan B	Laporan final rapi
C19	Menyusun script demo video	Alur demo maksimal 10 menit
C20	Menyiapkan release GitHub	Source code, executable jika ada, README, laporan

6.3 File yang Menjadi Tanggung Jawab C

Contoh struktur file:

```

client/main.py
client/cli/menu.py
client/services/vault_service.py
client/services/api_client.py
client/storage/local_storage.py
client/storage/backup_storage.py
client/models/password_entry.py
tests/test_end_to_end.py
README.md
docs/demo_script.md

```

6.4 Bagian Laporan yang Dikerjakan C

C menulis bagian:

1. Alur penggunaan aplikasi CLI.
2. Alur pembuatan vault.

3. Alur akses normal.
4. Alur mode backup.
5. Implementasi CRUD data password.
6. Penjelasan backup vault lokal.
7. Pengujian end-to-end.
8. Kesimpulan.
9. Lampiran repositori, video demo, dan pembagian tugas.
10. Penyatuan laporan dari A dan B.

6.5 Risiko Pekerjaan C

Risiko	Mitigasi
Integrasi crypto-server-CLI terlambat	C mulai membuat CLI skeleton sejak awal menggunakan mock function
Mode backup tidak berjalan karena backup vault tidak konsisten	Backup vault diperbarui setiap update mode normal
Testing terlalu banyak dan memakan waktu	Buat checklist testing sejak awal
Laporan tidak selesai H-1	A dan B wajib mengirim bagian laporan masing-masing sebelum H-2 malam

7. Dependency Antar Tugas

Tugas	Bergantung pada	Penanggung Jawab
CLI create vault	Crypto generate master key, AES-GCM, SSS	C bergantung pada A
Simpan server share	Format share dari A	B bergantung pada A
Open vault mode normal	API get vault dari B dan reconstruct key dari A	C bergantung pada A dan B
Open vault mode backup	Local share decrypt dan recovery share dari A	C bergantung pada A
CRUD password	Vault berhasil didekripsi dan dienkripsi ulang	C bergantung pada A
Update vault ke server	Endpoint update vault	C bergantung pada B
Backup vault lokal	AES-GCM dari A dan CLI storage dari C	C bergantung pada A
Test server zero-knowledge	Schema dan API selesai	B

Tugas	Bergantung pada	Penanggung Jawab
Test end-to-end	Modul A, B, C selesai	C, dibantu A dan B
Laporan implementasi	Semua modul selesai	Semua anggota
Video demo	Implementasi dan testing selesai	C sebagai integrator, dibantu A dan B

8. Kesepakatan Sebelum Implementasi

Sebelum coding dimulai, kelompok perlu menyepakati beberapa hal berikut.

8.1 Format Share

Format recovery share, local share, dan server share harus konsisten. Contoh format:

```
{
  "x": 1,
  "y": "base64_or_hex_value"
}
```

Pembagian:

- Share 1: local share
- Share 2: server share
- Share 3: recovery share

8.2 Format Vault Plaintext

Sebelum dienkrpsi, isi vault dapat berbentuk JSON:

```
{
  "entries": [
    {
      "id": "uuid",
      "nama_layanan": "GitHub",
      "username": "user@example.com",
      "password": "password_tersimpan",
      "catatan": "Akun utama"
    }
  ]
}
```

8.3 Format Data Terenkripsi

Ciphertext vault harus menyimpan:

- encrypted_vault
- nonce
- authentication tag jika library tidak menggabungkannya otomatis

8.4 Data yang Disimpan Client

Client menyimpan:

- encrypted local share
- nonce local share
- salt KDF
- parameter KDF
- backup vault terenkripsi
- nonce backup vault
- konfigurasi user lokal jika diperlukan

8.5 Data yang Disimpan Server

Server menyimpan:

- server share
- vault terenkripsi
- nonce vault
- metadata pengguna

Server tidak boleh menyimpan:

- master key utuh
- local share
- recovery share
- isi vault plaintext
- master password plaintext
- key turunan dari master password

8.6 Kontrak API

Minimal endpoint yang disepakati:

Method	Endpoint	Fungsi
POST	/vault/create	Membuat data vault baru di server
GET	/vault/{user_id}	Mengambil server share, encrypted vault, nonce
PUT	/vault/{user_id}	Memperbarui encrypted vault dan nonce
GET	/health	Mengecek server aktif

9. Timeline Pengerjaan

Target internal:

- Implementasi selesai H-3
- Testing selesai H-2
- Laporan selesai H-1
- Hari-H hanya untuk final check dan release

9.1 Timeline Detail

Waktu	Target	A	B	C
H-7	Persiapan awal	Finalisasi desain crypto	Finalisasi schema DB dan API	Finalisasi struktur CLI dan flow user
H-6	Implementasi dasar	AES-GCM, KDF, SSS dasar	Setup server dan SQLite	CLI skeleton dan local storage
H-5	Implementasi fitur utama	Rekonstruksi key, local share encryption	Endpoint create/get/update vault	Create vault dan open vault normal
H-4	Integrasi awal	CSPRNG dan validasi share	Validasi zero-knowledge server	CRUD password dan backup local
H-3	Implementasi selesai	Bonus QR + visual crypto selesai	Backend stabil	Mode backup dan integrasi end-to-end selesai
H-2	Testing selesai	Test crypto dan bonus	Test server dan database	Test end-to-end semua skenario
H-1	Laporan selesai	Kirim bagian teori dan crypto	Kirim bagian server dan DB	Gabung laporan, README, script demo
Hari-H	Finalisasi	Review akhir	Review akhir	Release GitHub dan cek link video

10. Checklist Implementasi

10.1 Checklist A — Crypto

- ☐ Generate master key acak.
- ☐ Bagi master key menjadi tiga share dengan SSS (2,3).
- ☐ Rekonstruksi master key dari dua share valid.
- ☐ Enkripsi vault dengan AES-128-GCM.
- ☐ Dekripsi vault dengan AES-128-GCM.
- ☐ Generate nonce unik setiap enkripsi vault.
- ☐ Derive key dari master password menggunakan KDF.
- ☐ Enkripsi local share.
- ☐ Dekripsi local share.
- ☐ Tolak akses jika master password salah.
- ☐ Generate password otomatis dengan CSPRNG.
- ☐ Format recovery share memuat koordinat dan nilai share.
- ☐ Recovery share dapat diubah menjadi QR code.
- ☐ QR code dibagi menjadi dua gambar share.
- ☐ Dua gambar share dapat digabung kembali menjadi QR code.
- ☐ QR hasil gabungan menghasilkan recovery share yang sama.

10.2 Checklist B — Backend dan SQLite

- ☐ Server dapat dijalankan.
- ☐ SQLite database terbentuk.
- ☐ Tabel user/metadata tersedia.
- ☐ Tabel vault tersedia.
- ☐ Server dapat menyimpan server share.
- ☐ Server dapat menyimpan vault terenkripsi sebagai BLOB.
- ☐ Server dapat menyimpan nonce vault.
- ☐ Server dapat mengirim server share, vault terenkripsi, dan nonce ke client.
- ☐ Server dapat menerima update vault terenkripsi dan nonce baru.
- ☐ Server tidak menerima master key.
- ☐ Server tidak menerima local share.
- ☐ Server tidak menerima recovery share.
- ☐ Server tidak menerima plaintext vault.
- ☐ Server tidak melakukan enkripsi/dekripsi vault.
- ☐ Endpoint health check tersedia.

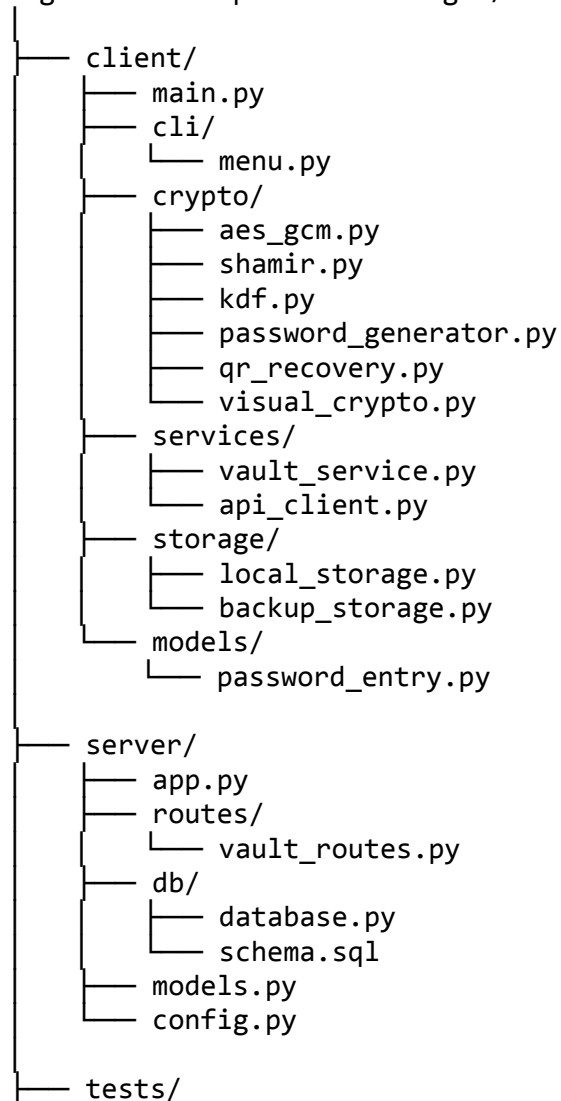
10.3 Checklist C — CLI dan Integrasi

- ☐ CLI memiliki menu utama.
- ☐ User dapat membuat vault baru.
- ☐ User dapat membuka vault mode normal.
- ☐ User dapat membuka vault mode backup.
- ☐ User dapat melihat data password.

- ☐ User dapat menambah data password manual.
 - ☐ User dapat menambah data password otomatis.
 - ☐ User dapat mengubah data password.
 - ☐ User dapat menghapus data password.
 - ☐ Mode backup hanya dapat melihat data.
 - ☐ Setelah perubahan, vault dienkripsi ulang.
 - ☐ Setelah perubahan, nonce baru dibuat.
 - ☐ Setelah perubahan, server diperbarui.
 - ☐ Setelah perubahan, backup vault lokal diperbarui.
 - ☐ Jika server mati, mode backup dapat digunakan.
 - ☐ Jika recovery share salah, akses ditolak.
 - ☐ Jika backup vault dimodifikasi, AES-GCM gagal melakukan dekripsi.
-
-

11. Struktur Repository yang Disarankan

tugas4-ii4021-password-manager/



```
|
|   |— test_crypto.py
|   |— test_server.py
|   |— test_end_to_end.py
|
|— docs/
|   |— demo_script.md
|   |— screenshots/
|
|— README.md
|— requirements.txt
|— NIM1_NIM2_NIM3_Tugas4_II4021.pdf
```

12. README Minimum

README harus memuat:

1. Nama dan deskripsi program.
 2. Anggota kelompok.
 3. Tech stack.
 4. Dependensi.
 5. Cara instalasi.
 6. Cara menjalankan server.
 7. Cara menjalankan client.
 8. Cara membuat vault.
 9. Cara membuka vault mode normal.
 10. Cara membuka vault mode backup.
 11. Cara menjalankan testing.
 12. Penjelasan singkat struktur folder.
 13. Catatan keamanan dan batasan implementasi.
-

13. Ringkasan Pembagian Tugas Final

Anggota	Tugas Utama	Tugas Tambahan
A	Crypto core, SSS, AES-GCM, KDF, CSPRNG, bonus QR dan visual cryptography	Unit test crypto, teori crypto, pengujian bonus
B	Server, API, SQLite, penyimpanan vault terenkripsi, server share, nonce, metadata	Test server, dokumentasi API, penjelasan zero-knowledge server

Anggota	Tugas Utama	Tugas Tambahan
C	CLI, user flow, CRUD password, mode normal, mode backup, backup lokal, integrasi end-to-end	README, laporan final, video demo, release GitHub

Dengan pembagian ini, A memegang bagian kriptografi yang kompleks, B memegang bagian server dan database yang penting untuk arsitektur client-server, dan C memegang bagian CLI serta integrasi akhir. Beban kerja dibuat seimbang karena setiap anggota memiliki modul teknis utama, pengujian masing-masing, serta kontribusi laporan sesuai bagian yang dikerjakan.